

Homework 3: Fixed-outline Floorplanning with Fixed and Soft Modules

Name: 陸明德

Student ID: 111062693

How to compile and execute program ?

How to Compile

In "HW3/src/", enter the following command to compile the program.

```
$ make
```

An executable file "hw3" will be generated in "HW3/bin/".

If you want to remove it, please enter the following command.

```
$ make clean
```

How to Run

Usage:

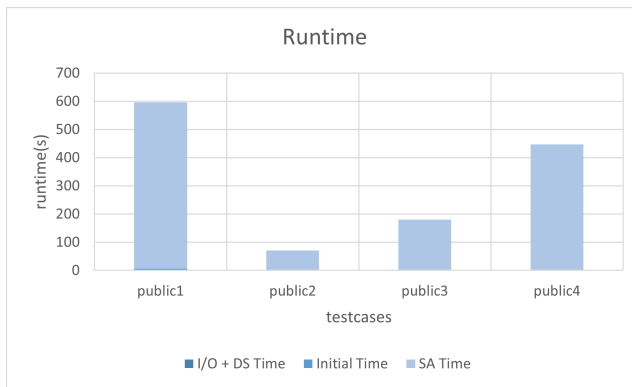
```
$ ./hw3 <text_file> <out_file>
```

Example: in "HW3/bin/", enter the following command:

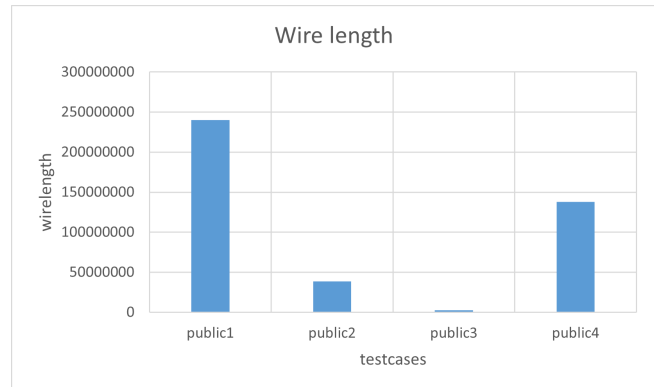
```
$ ./hw3 ../testcase/public1.txt ../output/public1.floorplan
```

The wirelength and the runtime of each testcase.

Testcase	I/O + DS Time	Initial Time	SA Time	Wire length
public1	0.000523	7.09146	590.001	239984392
public2	0.000575	0.024426	71.0823	38494434
public3	0.000561	0.310437	179.971	2621582
public4	0.000665	0.294962	446.354	137686350



runtime comparison



wirelength comparison

- I/O + DS Time means the time of reading the input file and constructing the data structures.
- Initial Time means the time of initial floorplanning.
- SA Time means the time of Simulated Annealing algorithm.

In the table, we can observe that the bottleneck of the program is the SA Time.

How did you determine the shapes of the soft modules? What are the benefits of your approach?

Since we know the mininum size and the aspect ratio (height/width) of each soft module.

We can calculate the width range at the mininum area by below equation:

```
int min_width = ceil(sqrt(min_area / 2));
int max_width = floor(sqrt(2 * min_area));
```

In some cases, the width range is too large.

In my experiment, if the width difference in two shape of the soft module is not considerable, the final result is almost the same.

So we only need at most 20 shapes in the range. We can calculate the step size by below equation:

```
int step = (max_width - min_width) / 20;
for(int i = min_width; i <= max_width; i += (step < 1 ? 1 : step)){
    this->shape.push_back(make_pair(i, ceil((float)min_area / i)));
}
```

The benefit of this approach is that the area of each soft module is close to the mininum area.

Second, since the shape is at most 20, we can speed up the process of the placement when we try to find the best shape for each soft module in constant time.

The details of your floorplanning algorithm.

The algorithm is as below:

1. Read the input file and construct the data structures.
2. Initial floorplanning.
3. Simulated annealing algorithm.
4. Write the result to the output file.

Constructing data structures

Insted of using common P-admissible floorplan representation like b*-tree or sequence pair, I use the grid-based floorplan representation. Each position in the grid is a unit of the chip.

The grid is a 2D vector of boolean. If the position is occupied, the value is true. Otherwise, the value is false.

For each module, I define a struct to store the information of the module. The struct contains the width, height, x, y, and the vector of the shape of the soft module.

Furthermore, There is a set of temp_x, temp_y, temp_width, temp_height to store the temporary position of the module. This is used in SA algorithm to recover the position and shape of the module when the module is not accepted.

Initial floorplanning

At first, I sort the soft modules by the area in descending order.

For each soft module, we first find a free position in the chip.

In order to find a free position, starting from the root (0, 0) each time move a random step to the right until the right boundary of the chip. then move a random step to the top until the top boundary of the chip.

In the loop, if the position is free, we place the soft module at the position by trying all the shapes of the soft module. If it can be placed, we place it and break the loop. Otherwise, we try the next shape.

If all the shapes of the soft module cannot be placed, we move to the next position. Once the soft module is placed, we compact the chip by moving the soft module to the leftmost and the downmost position.

```
bool placed_block(block& module, vector<vector<bool>>& Grid){
    for(int y = 0; y < Grid.size(); y += (Grid.size() < 20 ? 1 : rand() % 11 + 20)){
        for(int x = 0; x < Grid[0].size(); x += (Grid[0].size() < 20 ? 1 : rand() % 11 + 20)){
            if(!Grid[y][x]){
                for(auto it: module.shape){
                    if(check_placed(x, y, it.first, it.second, Grid)){
                        pair<int, int> new_pos = compact(x, y, it.first,
it.second, Grid);
                        module.x = new_pos.first;
                        module.y = new_pos.second;
                        module.width = it.first;
                        module.height = it.second;
                        set_grid(Grid, module.x, module.y, module.width,
module.height, true);
                    }
                }
            }
        }
    }
}
```

```

        return true;
    }
}
}
}
return false;
}

```

if the soft module cannot be placed, we reset the grid and try again at the beginning of the loop.

Computing parts

I implement the Simulated Annealing algorithm in the paper "A New Algorithm for Floorplan Design" by D.F. Wong and C.L. Liu.

The algorithm is as below:

```

while((double)reject_cnt / gen_cnt <= REJECT_RATIO && T >= T_MIN && ((clock() -
start_time) / CLOCKS_PER_SEC <= 590)) {
    // Initialize
    gen_cnt = 0, uphill_cnt = 0, reject_cnt = 0;

    // Generate neighbor
    while(uphill_cnt <= N && gen_cnt <= DOUBLE_N && ((clock() - start_time) /
CLOCKS_PER_SEC <= 590)) {
        int r = rand() % 100;
        // changed for restore
        vector<block*> changed = gen_neighbor(SoftModuleVector, FixedModuleVector,
Grid, r);
        int neighbor_cost = cost_function(ChipSizeWidth, ChipSizeHeight, alpha,
beta, SoftModuleVector, FixedModuleVector, NetVector, wirelength);
        gen_cnt++;

        int delta_cost = neighbor_cost - cost;
        bool rand_accept = (double)rand() / RAND_MAX < exp(-1 * (delta_cost) / T);
        if(delta_cost < 0 || rand_accept) {
            if(delta_cost > 0) {
                uphill_cnt++;
            }
            cost = neighbor_cost;
            if(cost < min_cost) {
                min_cost = cost;
                // update best solution
                best_sol.clear();
                for(auto it: SoftModuleVector){
                    block b(it->name, it->x, it->y, it->width, it->height);
                    best_sol.push_back(b);
                }
                best_wirelength = wirelength;
            }
        }
    }
}

```

```

    } else {
        reject_cnt++;
        // restore SoftModuleVector value and Grid
        restore(changed, Grid);
    }

}

// Reduce temperature
T *= T_DECAY;
}

```

There is two main difference between my implementation and the paper.

1. Generate solution

First, I use the grid-based floorplan representation and the problem is non-slicing floorplan. So I used my own implementation of the operation to generate the neighbor(means the new solution).

There are three operation in my implementation.

1. Swap the positions of two randomly selected modules.
2. Move a randomly selected module to right or left by a random step.
3. Change the shape of a randomly selected module.

Operation two and three are easy to implement. Operation one is a little bit tricky.

In operation one, we need to check if the two modules are overlapped after the swap. Check the overlap have to be done in two steps. First, we set the grid of the two modules to false. Then we check if the second module can be placed at the position of the first module. If it can be placed, we place the second module at the position of the first module and check if the first module can be placed at the position of the second module. If it can be placed, we place the first module at the position of the second module. Otherwise, we restore the grid and return false.

```

bool swap_module(block& module1, block& module2, vector<vector<bool>>& Grid){
    // check if after swapping, module is overlapping with any of the modules
    set_grid(Grid, module1.x, module1.y, module1.width, module1.height, false);
    set_grid(Grid, module2.x, module2.y, module2.width, module2.height, false);
    // two step check
    if(check_placed(module1.x, module1.y, module2.width, module2.height, Grid)){
        // place module2 at module1's position
        set_grid(Grid, module1.x, module1.y, module2.width, module2.height, true);
        // after placing, check if module1 can be placed at module2's original
position
        if(check_placed(module2.x, module2.y, module1.width, module1.height,
Grid)){
            // if true, swap the modules
            set_grid(Grid, module2.x, module2.y, module1.width, module1.height,
true);
            module1.backup();
            module2.backup();

```

```

        swap(module1.x, module2.x);
        swap(module1.y, module2.y);
        return true;
    }
    else{
        // if not, restore the grid
        set_grid(Grid, module1.x, module1.y, module2.width, module2.height,
false);
    }
}
set_grid(Grid, module1.x, module1.y, module1.width, module1.height, true);
set_grid(Grid, module2.x, module2.y, module2.width, module2.height, true);
return false;
}

```

2. Cost function

Second, I use different cost function. The cost function I used is as below:

```

float cost_function(int ChipSizeWidth, int ChipSizeHeight, float alpha, float
beta, vector<block*> &SoftModuleVector, vector<block*> &FixedModuleVector,
vector<pair<int, pair<block*, block*>>>& NetVector, int& wirelength){
    int total_area = 0;
    for(auto it: SoftModuleVector){
        total_area += it->width * it->height;
    }
    for(auto it: FixedModuleVector){
        total_area += it->width * it->height;
    }
    wirelength = HPWL(NetVector);
    float cost = alpha * (float)total_area + beta * (float)wirelength;
    return cost;
}

```

Because our goal is to minimize the total area and the wirelength, we can use the linear combination of the total area and the wirelength as the cost function.

Through adjusting the alpha and beta, we can control the importance of the total area and the wirelength.

For fixed outline constraint, Every time we generate a new solution, we check if the new solution is within the fixed outline. If not, we reject the new solution.

Try your best to enhance your solution quality.

1. Initial floorplanning method

To avoid some situation that the soft module cannot be placed, I place the soft module by the order of the mininum area in descending order.

However, I am wondering if this approach is better than random order. Because the soft module with the maximum area may be placed at the position that is not good and in simulation annealing algorithm, it is hard to change its position since I compact the chip after placing the soft module.

testcase	before wirelength	before runtime	after wirelength	after runtime
public1	239984392	580.19	234745752	580.17
public2	38494434	64.05	33376867	69.61
public3	2621582	158.47	3141369	173.23
public4	137686350	410.98	146237950	442.54

The result shows that random order is not always better than the order of the minimum area in descending order.

2. Generate solution method

I am curious about the three operations I used to generate the solution.

I think changing the shape of the soft module is not a good operation. Because the shape of the soft module is already close to the minimum area. Changing the shape of the soft module does not help to reduce the cost.

So, I remove the operation to change the shape of the soft module. Only two operations are left.

testcase	before wirelength	before runtime	after wirelength	after runtime
public1	239984392	580.19	240874222	580.19
public2	38494434	64.05	38527745	64.53
public3	2621582	158.47	3271642	179.05
public4	137686350	410.98	138332850	414.72

The result shows that removing the operation to change the shape of the soft module does not help to reduce the wirelength.

What have you learned from this homework?

I learned how to implement the Simulated Annealing algorithm and how to use it to solve the fixed-outline floorplanning problem.

It's interesting that some floorplan representation like b*-tree is not suitable for this floorplanning problem. Because the hard modules are fixed, we cannot change the position of the hard modules. B*-tree is not suitable for this problem because it is hard to find the root of the b*-tree.

The biggest problem I encountered is that the SA algorithm is too slow by Grid-based floorplan representation.

Since every time we generate a new solution, we want to create a temporary grid to check if the new solution is better than the current solution and we don't want to change the current grid.

The process to create a temporary grid is too slow. So I use the vector of the pointer to soft module that indicates the changed soft module in the new solution. This approach is much faster than creating a temporary grid.

Reference

- [A New Algorithm for Floorplan Design](#)